

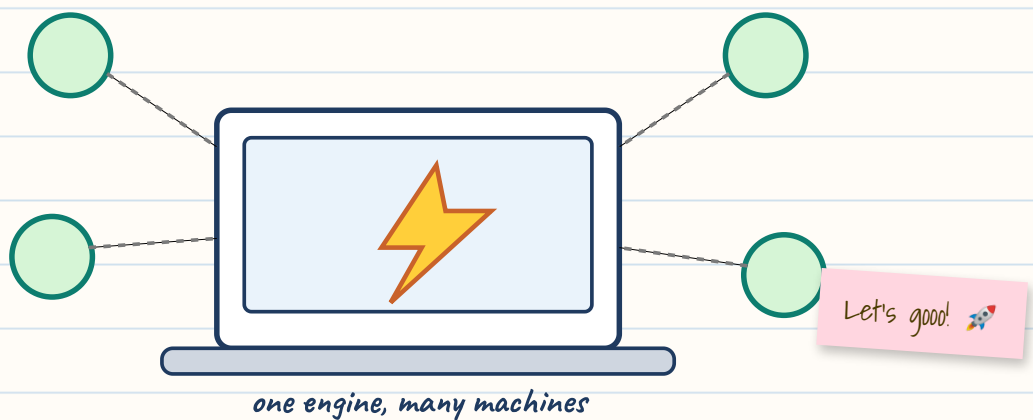
4 Vs, Hadoop,
Spark stack,
Driver & Executors...

◆ My Study Notes ◆

PySpark

Day 1 Notebook

Big Data Landscape & Introduction to Apache Spark



Contents: Big Data & 4Vs · pandas limits · Hadoop ecosystem · Why Spark · Spark stack · Architecture ·
Where PySpark fits · Case studies · Glossary · Practice

cover

Topic 1

What is "Big Data"?

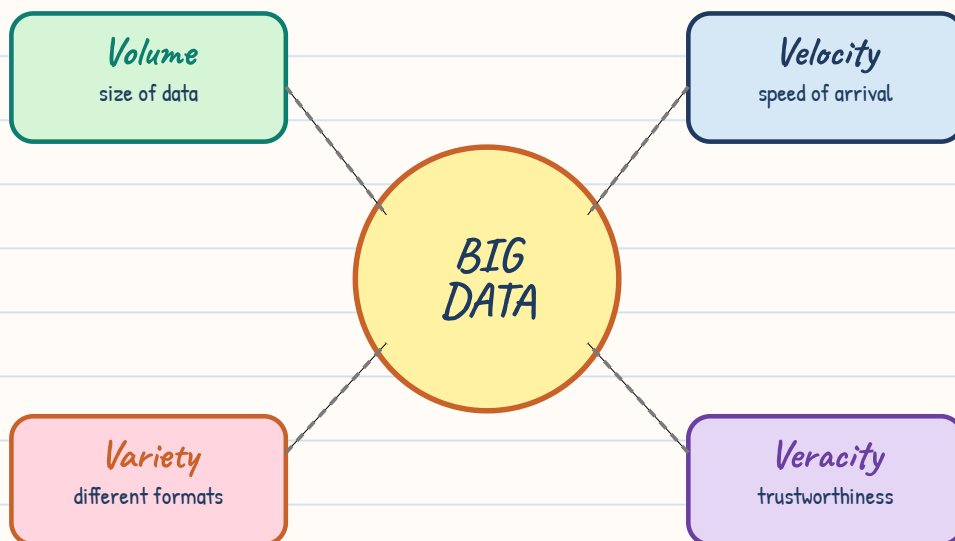
...and the famous 4 V's 📌

Big data = data that is too **big**, too **fast**, or too **messy** for a single computer / normal tools (like Excel or pandas) to handle comfortably.

Why does this even matter?

Because once data crosses a certain size or speed, our usual tricks (loading a CSV into RAM) simply stop working — we need many machines working *together*. That's the whole reason Hadoop and Spark exist!

The 4 V's



- **Volume** — the sheer amount of data (GBs → TBs → PBs). E.g. a large social network can generate several **petabytes per day**.
- **Velocity** — how fast new data pours in. E.g. stock ticks, IoT sensors, app clickstreams arriving every second.
- **Variety** — many formats: structured (tables), semi-structured (JSON/XML), unstructured (images, video, free text).

- **Veracity** — is the data even reliable? Real data has duplicates, missing values, bots, typos.

Example — Netflix: Volume = watch-history of 200M+ users. Velocity = real-time click/play events. Variety = video files + star ratings + text reviews. Veracity = fake reviews / incomplete watch data that must be cleaned.

More examples, one V at a time

- **Volume:** YouTube users upload roughly **500 hours of video every minute** — no single hard drive or database was ever meant to hold that.
- **Velocity:** During a cricket/football final, millions of tweets can fire off **per minute** — the system has to ingest and react before the moment is even over.
- **Variety:** A hospital's records include structured lab-test tables, semi-structured HL7/JSON messages, *and* unstructured doctor notes and X-ray images — all describing the same patient.
- **Veracity:** A retailer's sensor-based footfall counter might double-count people who pause near the door — the raw number looks precise but isn't necessarily true.

A simple gut-check

Ask yourself: "Can one good laptop store this, process it in a reasonable time, and can I trust the numbers as-is?" If the honest answer to any part is **no**, you've stepped into "big data" territory — and that's exactly the situation Spark was designed for.

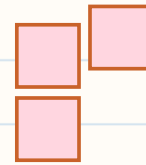
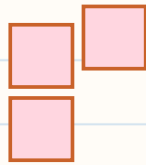
page 1

Topic 2

Why pandas struggles

the "single machine" problem 

pandas is amazing for small-medium data — but it was built to run on **one computer, using one machine's RAM**.



too much data overflowing the RAM = MemoryError 💥

- pandas loads the **entire dataset into RAM**. Bigger than your RAM → crash (MemoryError).
- Mostly **single-threaded** – it barely uses your extra CPU cores, let alone other machines.
- **No fault tolerance** – if the script or machine dies mid-run, you lose everything and start over.

pandas vs Spark – when to use which?

Situation	pandas	Spark / PySpark
Data size	fits in RAM (MB–few GB)	too big for one machine (GBs–TBs+)
Machines used	1	many, in a cluster
Speed on huge data	slow / crashes	fast (parallel + in-memory)
Fault tolerance	none	built-in (recomputes lost data)
Best for	quick exploration, small ML	production pipelines, huge datasets

Rule of thumb 👍

If your laptop can open it comfortably → use pandas. If it needs a cluster, or must never lose progress halfway → reach for Spark.

A worked example

Scenario: you have a 200 GB CSV of e-commerce orders and need to compute total revenue per city.

- With **pandas** – `pd.read_csv()` tries to pull all 200 GB into RAM at once. A typical laptop (8–16 GB RAM) simply cannot hold it → MemoryError before you even start computing.
- With **PySpark** – the file is automatically split into partitions, spread across many machines, and each machine only holds and processes its own slice. The final "sum per city" is combined at the end.

pandas isn't "bad" – it's just built for a different job

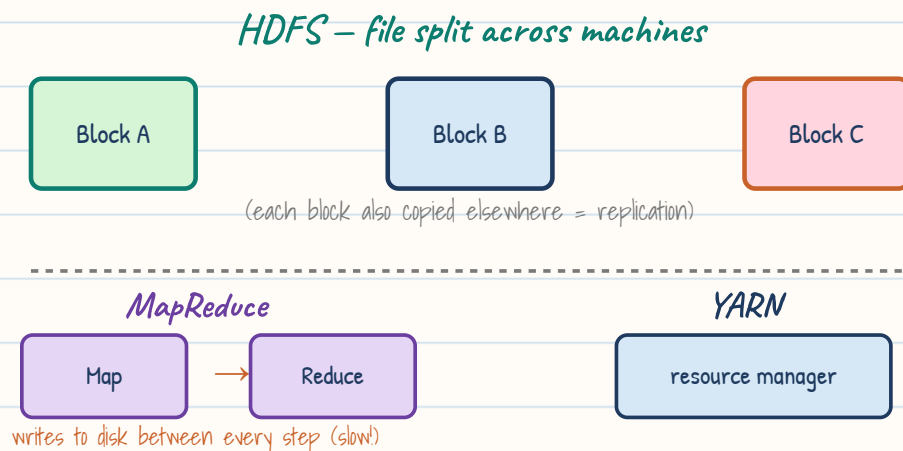
For fast exploration of a CSV that fits on your laptop, pandas is actually *faster* to write and iterate with than Spark (no cluster setup, no distributed overhead). Many real workflows start in pandas, and only move to PySpark once the data outgrows one machine.

Topic 3

The Hadoop Ecosystem

the framework that came before Spark 🐘

Hadoop (2006) was the **original** big-data framework. It has 3 main pieces:



- **HDFS (Hadoop Distributed File System)** – splits big files into blocks (default 128 MB each), spreads them across many machines, and keeps replica copies (default **3 copies**) so no data is lost if one machine dies.
- **MapReduce** – a 2-step model: **Map** processes chunks in parallel, **Reduce** combines the results. Downside: writes intermediate results **to disk** at every step → slow.
- **YARN (Yet Another Resource Negotiator)** – the "manager" that decides which machine runs which job, and how much CPU/RAM it gets. Made of a **ResourceManager** (the master, tracks the whole cluster) and a **NodeManager** on every machine (reports its own free resources).

Worked example – Word Count (the "hello world" of MapReduce):

Input: three text files spread across the cluster.

1. **Map step** – each machine reads its own chunk and emits (word, 1) pairs for every word it sees, in parallel.
2. **Shuffle** – all pairs with the same word get grouped together (written to disk here – this is the slow part!).
3. **Reduce step** – for each word, add up all the 1's to get its final total count.

Key takeaway

Hadoop solved the "too big for one machine" problem — but MapReduce's constant disk writing made it slow, especially for jobs that repeat many times (like ML training). That gap is exactly what Spark was built to fix. 🖱️ Next page!

page 3

Topic 4

Why Apache Spark?

in-memory computing changes everything ⚡

Spark's core insight: instead of writing intermediate results to disk every step (like MapReduce), keep the data in RAM across operations.

MapReduce (disk-based)



disk read/write every single step 🐢

Spark (in-memory)



up to 10-100x faster for repeated/iterative work

- **In-memory computing** — data stays in RAM between operations instead of round-tripping to disk.
- **Speed** — especially huge for *iterative* algorithms (machine learning training loops, graph algorithms) that touch the same data again and again.
- **Unified engine** — one framework for batch processing, SQL, streaming, ML, and graphs — instead of stitching together 5 separate tools.

Analogy: MapReduce is like re-reading your notes from a locker every single time you need them. Spark is like keeping your notes open on the desk in front of you while you work.

The idea behind the speed: RDDs

Spark's original in-memory building block is called an **RDD** (Resilient Distributed Dataset) — think of it as a collection of data spread across the cluster that Spark remembers *how to rebuild* if a machine fails, instead of needing a saved copy on disk. "Resilient" = it can recover itself; "Distributed" = it lives across many machines.

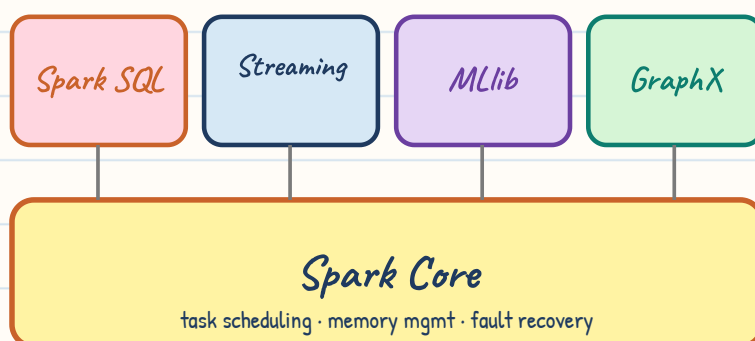
Iterative example — training a model: Logistic regression training loops over the same dataset dozens of times, tweaking weights each round. With MapReduce, each round means writing results to disk and reading them back — 20 rounds = 20 rounds of disk I/O. With Spark, the dataset is cached in RAM once, and every round just reuses it in memory. This is the exact scenario where Spark's own benchmarks showed roughly **100x** speedups over MapReduce.

page 4

Topic 5

Spark's Component Stack

one engine, five specialised libraries 🧱



everything sits on top of Spark Core — the foundation

- **Spark Core** — the engine underneath everything: schedules tasks, manages memory, recovers from failures. This is the foundation.
- **Spark SQL** — run SQL queries or use DataFrames to work with structured data.
- **Spark Streaming** (Structured Streaming) — process live, continuously-arriving data (e.g. sensor feeds, logs).
- **MLlib** — built-in machine learning algorithms: classification, clustering, recommendation, etc.

- **GraphX** — analyse graph-shaped data — social networks, recommendation graphs, network routes.

Why "unified" matters

You don't need 5 different tools glued together — one Spark cluster can run a SQL query, train an ML model, and process a data stream, all sharing the same engine underneath.

One example per library

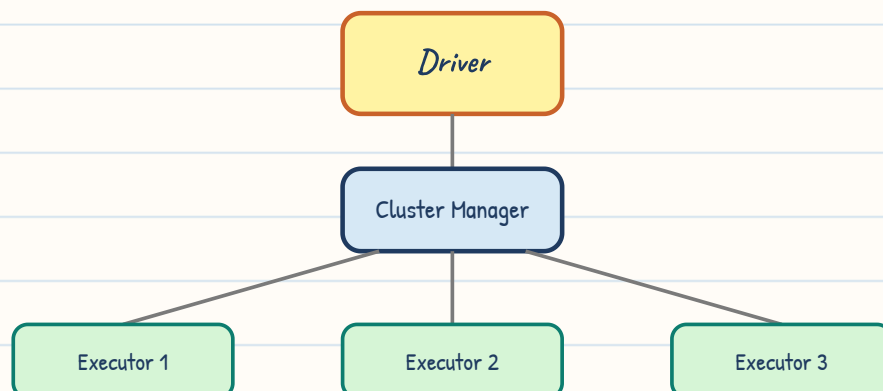
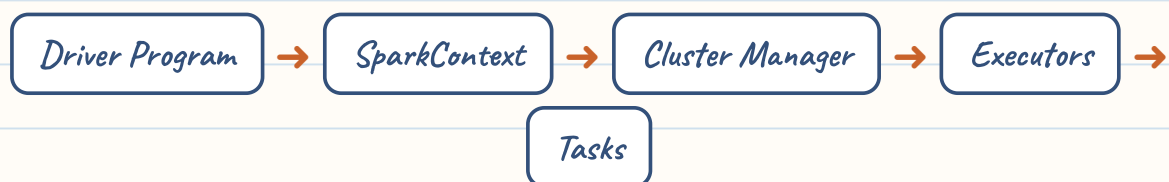
Library	Used for	Example
Spark SQL	querying structured data	find average order value per region from a sales table
Streaming	live, continuous data	flag a bank transaction as suspicious the moment it happens
MLlib	machine learning at scale	train a churn-prediction model on 50M customer records
GraphX	graph/network data	find the shortest path between two people in a social network

page 5

Topic 6

Spark Architecture

Driver → SparkContext → Cluster Manager → Executors → Tasks 🏠



each executor runs many small Tasks on its slice of the data

- **Driver Program** — the "project manager." Your actual code runs here; it plans the whole job.
- **SparkContext** — the driver's phone line to the outside world; connects it to the cluster.
- **Cluster Manager** — the "HR department"; decides which machines are free and assigns them (options: standalone, YARN, Kubernetes...).
- **Executors** — the actual workers on each machine; they run tasks and hold data in memory.
- **Tasks** — the smallest to-do items, handed out to executors to work on in parallel.

Walking through a real run

You write: `df.filter(...).groupBy("city").sum("revenue")`

1. The **Driver** breaks this into a **Job**, then splits the job into **Stages** (groups of steps that can run without needing to reshuffle data).
2. Each Stage is split into many small **Tasks** — one Task per data partition.
3. The **Cluster Manager** hands Tasks to available **Executors**, which run them in parallel and hold intermediate results in memory.
4. The Driver collects and combines the final results back for you.

Analogy recap

Driver = project manager writing the plan · Cluster Manager = HR assigning available workers ·
Executors = the workers on-site · Tasks = individual bricks each worker lays. Nobody touches
the whole wall alone — everyone builds their section in parallel.

page 6

Topic 7

Where does PySpark fit?

Python's doorway into Spark  

Spark itself is written in **Scala** and runs on the **JVM** (Java Virtual Machine). **PySpark** is simply the **Python API** for it.



- You write familiar **Python** code (DataFrames, functions, etc.)
- A bridge called **Py4J** translates your calls into instructions for the JVM-based Spark engine.
- Spark then distributes the actual heavy-lifting across the cluster — the computation still runs at full JVM speed.

Why PySpark specifically?

Python is beginner-friendly and has a massive data-science ecosystem (pandas, numpy, scikit-learn). PySpark lets you keep that comfort while getting Spark's distributed power underneath — best of both worlds.

Mental model: pandas = doing your homework alone at your desk. PySpark = you write the same homework instructions, but a whole classroom of students (executors) does the work in parallel, and you just collect the combined answer.

Your very first PySpark line (preview)

SparkSession — your entry point

Every PySpark program starts by creating a **SparkSession** — this is what sets up the Driver, talks to SparkContext, and connects to the cluster behind the scenes:

```
spark = SparkSession.builder.appName("MyFirstApp").getOrCreate()
```

Everything you do after this — reading files, running SQL, building DataFrames — flows through `spark`. We'll actually run this on Day 2!

Bonus

Real-World Case Studies

why companies actually reach for Spark 🏢

🚗 **Uber** — processes trillions of GPS/trip events to power real-time pricing (surge pricing), ETA predictions, and fraud detection. Data arrives continuously from millions of devices — a single machine could never keep up with that velocity.

🛒 **Amazon** — analyses huge volumes of clickstream and purchase data to power "customers who bought this also bought..." recommendations, refreshed constantly as new orders come in across the globe.

🎬 **Netflix** — runs Spark jobs across petabytes of viewing data to personalise recommendations and decide what content to produce next, blending structured ratings with unstructured viewing behaviour.

🏠 **Banks / Fintech (e.g. fraud teams)** — score transactions for fraud risk within milliseconds using streaming pipelines, while also running huge nightly batch jobs to retrain fraud-detection models on historical data.

The common thread 🧵

In every case, the data is too big for one machine (Volume), arrives too fast to batch up and wait (Velocity), or needs the same heavy computation repeated constantly (retraining models, recomputing recommendations). That combination is exactly what distributed, in-memory processing like Spark was built for.

Bonus

Quick Glossary

key terms, all in one place 📖

Term	Plain-English meaning
Cluster	a group of machines working together as one system
Node	a single machine within the cluster
Partition	a chunk of the dataset stored/processed on one node
RDD	Spark's original in-memory, fault-tolerant distributed collection
DataFrame	a table-like structure built on top of RDDs, with named columns (like a pandas DataFrame, but distributed)
Driver	the process running your main code, plans the whole job
Executor	a worker process that runs tasks and holds data in memory
Job	the full unit of work triggered by an action (like <code>.count()</code>)
Stage	a group of tasks that can run without reshuffling data
Task	the smallest unit of work, run on one partition by one executor
Fault tolerance	the system's ability to recover automatically if a machine fails

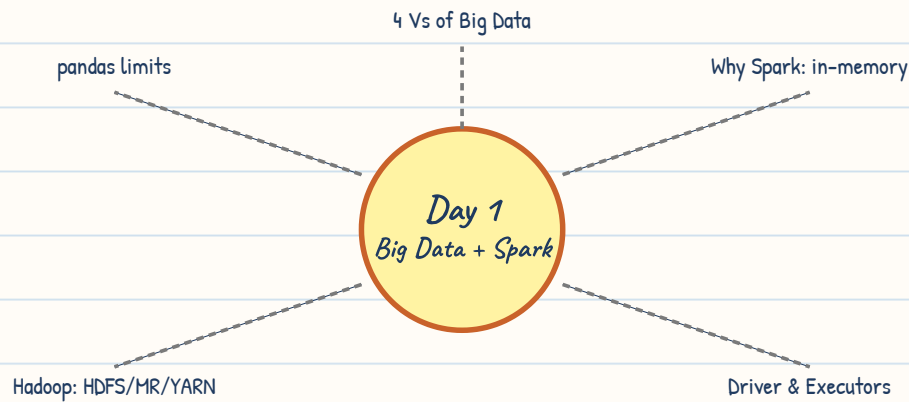
Keep this page bookmarked 📌

These terms will come up constantly from Day 2 onward once we start writing real PySpark code — refer back here whenever a word feels unfamiliar.

Wrap-up

Day 1 Recap + Practice

quick mind-map + your to-dos 



Practice Exercises

- ☐ Write a short paragraph comparing pandas vs Spark — when would you reach for each?
- ☐ List 3 real-world companies/use-cases where distributed processing is necessary.
- ☐ Draw the Spark cluster architecture diagram from memory (Driver → SparkContext → Cluster Manager → Executors → Tasks).
- ☐ Bonus: pick one company from the case-studies page and identify which of the 4 V's applies most to their problem, and why.

Say it out loud (hands-on exercise)

In one sentence each, explain: What does the Driver do? What does SparkContext do? What does the Cluster Manager do? What do Executors do? What is a Task?

? Common beginner questions

"Do I need a real cluster to learn PySpark?" No — PySpark can run in "local mode" on a single laptop, simulating multiple executors as threads. Perfect for learning before touching a real cluster.

"Is Spark replacing pandas entirely?" Not really — they solve different problems. Many teams use pandas for small/medium analysis and switch to PySpark only once data outgrows one machine.

Tomorrow (Day 2): RDDs + DataFrames — actually writing PySpark code! 🎉